

Product Requirement Document (PRD)

Mobile Price-Range Classifier with Feature Selection

— evolved into an Assisted Pricing Optimization Engine —

1. Document Overview & Metadata

Field	Description / Student Inputs
Project Title	Mobile Price-Range Classifier with Feature Selection — Assisted Pricing Optimization Engine
Project Track	Advanced Data Science
Team Members & IDs	Name: Saloni Kumari & ID: 240410700059
Assigned Mentor	Soumen Mukherjee
Document Version & Date	v4.0 05-09-2026

2. Executive Summary & Problem Statement

2.1 Problem Statement

Product managers and pricing analysts in the Consumer Electronics / Retail space must position new mobile handsets into the correct price tier before launch, but doing so today relies heavily on manual competitor benchmarking and gut-feel comparisons of specifications. This is slow, inconsistent across analysts, and does not scale as the number of specifications (RAM, battery, camera, connectivity, etc.) grows. Existing spreadsheet-based comparisons cannot systematically identify which specifications actually drive price-tier placement, nor let an analyst ask “what if we changed this one component?” without rebuilding the comparison from scratch.

This project addresses that gap in two layers. The foundation (grounded in “An Introduction to Variable and Feature Selection,” JMLR 2003) uses Recursive Feature Elimination combined with a Random Forest classifier to predict a phone's price tier and rank which specifications matter most, on the Mobile Price Classification dataset. Building on that foundation, the project is scoped as an Assisted Pricing Optimization Engine (Chapter 5): TreeSHAP explains individual predictions, and an interactive what-if playground lets a PM drag specification sliders and see, in real time, how a trade-off shifts the predicted tier and an estimated margin.

2.2 Why This Goes Beyond a Kaggle Benchmark

A standard Kaggle submission stops at a leaderboard metric. This PRD scopes the project as a decision-support tool: every prediction is explainable (Chapter 14), every architectural choice is recorded and justified (Chapter 16), every business need traces to a shipped feature and test (Chapter 20), and the system is designed to be operated, not just trained once and demoed (Chapters 17–18).

3. Goals, Objectives & Success Metrics

- **Primary technical goal:** Build a Recursive Feature Elimination + Random Forest classifier-based system that predicts a mobile phone's price range (of 4 tiers) from its specifications on the Mobile Price Classification dataset.
- **Enhanced functional goal:** Package the trained pipeline as an Assisted Pricing Optimization Engine — a scikit-learn + TreeSHAP model served via FastAPI/PostgreSQL, consumed by a React dashboard with an interactive what-if slider playground — so PMs can check price-tier predictions, see why the model made them, and simulate spec trade-offs without writing code.
- **Committed success targets:** Accuracy $\geq 90\%$ and macro F1-score ≥ 0.88 across the four price-range classes, evaluated on the untouched 20% stratified test split (Chapter 12). Checkpointed at Viva 2; if not met, the fallback plan in Chapter 22 applies rather than silently lowering the bar.
- **Baseline for comparison:** A full-feature Random Forest classifier (no RFE) is trained as the baseline; the RFE-reduced model must match or exceed baseline Accuracy/macro F1 using materially fewer features, or the feature-selection claim is not considered validated.
- **Target audience:** Product managers and pricing analysts in the Consumer Electronics / Retail industry who need a fast, explainable, specification-driven view of where a device should be priced and how flexible that placement is.
- **Business impact:** Faster, more consistent price-tier decisions; a ranked feature-importance report and per-prediction SHAP explanation; and a what-if simulator that turns a static classifier into an interactive pricing-strategy tool.

4. Scope & User Personas

4.1 In Scope

- RFE + Random Forest price-tier classifier trained on the Mobile Price Classification dataset.
- FastAPI backend with the 5 endpoints in Chapter 11, backed by PostgreSQL.
- TreeSHAP local explainability and a React what-if slider playground with an illustrative margin estimate.
- Single-role (analyst) JWT authentication.
- Full documentation set: Model Card, ADRs, traceability matrix, README.

4.2 Out of Scope (this iteration)

- Multi-user roles, self-service signup, or password reset (Chapter 10.4).
- Real bill-of-materials/vendor cost data for margin estimation — the cost model is illustrative (Chapter 5, 13).
- PostGIS-backed regional price benchmarking — schema is reserved but not populated or queried (Chapter 9, ADR-003).
- Continuous/online retraining — the model is trained offline and versioned manually via `model_registry`.

4.3 Personas

- Priya, Product Manager: Needs to know, before a device spec is finalized, which price tier it lands in and which one or two spec changes would move it up or down a tier — without asking a data scientist each time.
- Arjun, Pricing Analyst: Needs a ranked, evidence-backed list of which specifications drive pricing across the whole portfolio, to inform category-level pricing strategy rather than a single device.

4.4 Representative User Stories

1. As Priya, I want to enter a candidate spec and see its predicted price tier, so that I can sanity-check positioning before it goes to engineering.
2. As Priya, I want to see why the model predicted that tier for this specific device, so that I can defend or challenge the recommendation in a stakeholder meeting.
3. As Priya, I want to drag a RAM or battery slider and see the tier and estimated margin update live, so that I can explore trade-offs interactively instead of re-running a spreadsheet.
4. As Arjun, I want a ranked list of which specifications most influence price tier across the dataset, so that I can prioritize which components matter most for the next product line.

5. Product Enhancement: Assisted Pricing Optimization Engine

Per mentor feedback, the project is elevated beyond a standard classification benchmark by pairing the RFE + Random Forest classifier with local explainability and an interactive simulation layer.

5.1 TreeSHAP Local Explainability

Every prediction served through `/predict` is explainable on demand via `/explain/{prediction_id}` (Chapter 11). TreeSHAP is used rather than a model-agnostic method (LIME, KernelSHAP) because it has an exact, polynomial-time algorithm for tree ensembles like Random Forest — see ADR-004 (Chapter 16) for the full comparison. The React dashboard renders the result as a waterfall chart showing which specifications pushed the prediction toward or away from each tier.

5.2 Interactive What-If Specification Playground

The React UI exposes sliders for the RFE-selected specifications (e.g., RAM, battery power, primary camera, internal storage). Moving a slider calls `POST /whatif` (debounced client-side) with the adjusted spec; the backend re-runs inference and returns the new predicted tier, the shift in class probabilities, and an estimated margin delta — enabling a PM to answer “what happens if I upgrade RAM but downgrade the camera?” in real time.

5.3 Illustrative Cost/Margin Model

Because real component-cost (BOM) data is not available in the Mobile Price Classification dataset, the margin estimate uses a simple, transparent, per-specification cost function (e.g., a linear cost-per-GB-of-RAM, cost-per-mAh-of-battery, calibrated to publicly known component price ranges) combined with the predicted tier's typical price midpoint. This is explicitly labeled in the UI and the Model Card (Chapter 13) as an illustrative approximation for demonstrating the trade-off concept — not a real pricing/margin authority.

5.4 New API Surface Introduced

- `POST /whatif` — accepts a baseline `prediction_id` plus one or more adjusted fields; returns the new tier, probability shift, and margin delta.
- `GET /explain/{prediction_id}` — returns TreeSHAP attribution values for a logged prediction.

6. System Architecture & High-Level Design

6.1 End-to-End System Pipeline

- **Data Ingestion:** Batch ingestion of the static Mobile Price Classification dataset (tabular CSV); no streaming requirement since the dataset is fixed and pre-labeled.
- **Processing & Modeling Engine:** A scikit-learn pipeline that cleans/scales features, runs RFECV wrapped around Random Forest per the CV schema (Chapter 12), and trains the final classifier.
- **Explainability Layer:** A TreeSHAP explainer loaded alongside the model, invoked on-demand via `/explain`.
- **Storage Layer:** PostgreSQL (predictions, what-if history, model registry, users) plus versioned dataset/model artifact files on disk (Chapter 8–9).
- **Presentation Layer:** A React (Tailwind + Axios) dashboard: prediction view, SHAP waterfall chart, and the what-if slider playground.

6.2 Architecture Diagram

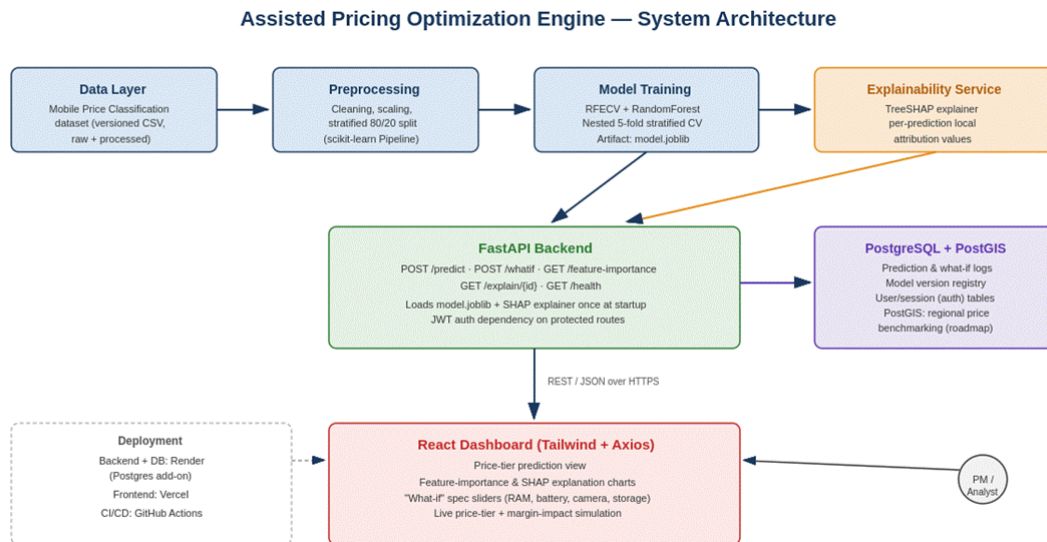


Figure 1 — Assisted Pricing Optimization Engine system architecture: data/model pipeline, FastAPI + PostgreSQL/PostGIS backend, and the React dashboard.

7. Low-Level Design & Component Specifications

7.1 Backend Module Layout

```
app/
  main.py          # FastAPI app, router registration, startup model
load
  api/
    predict.py     # POST /predict
    whatif.py      # POST /whatif
    explain.py     # GET /explain/{id}
    importance.py  # GET /feature-importance
    health.py      # GET /health
  services/
    inference.py   # loads model.joblib, runs predictions
    shap_service.py # TreeSHAP explainer, waterfall payload builder
    margin_model.py # illustrative cost/margin calculation (Ch.
5.3)
  db/
    models.py      # SQLAlchemy models for the 5 tables (Ch. 9)
    session.py     # PostgreSQL connection/session management
  auth/
    jwt_handler.py # token issue/verify, scoped to /predict,
/whatif, /explain
  schemas/
    prediction.py  # Pydantic request/response models (Ch. 11)
```

7.2 Frontend Component Layout

- **PredictionForm** — collects specification inputs, calls `/predict`.
- **FeatureImportanceChart** — renders `/feature-importance` as a horizontal bar chart.
- **ShapWaterfall** — renders `/explain/{id}` as a waterfall chart of attribution values.
- **WhatIfSliders** — one slider per RFE-selected numeric feature; debounced calls to `/whatif`.
- **MarginImpactPanel** — displays the tier + probability shift + margin delta returned by `/whatif`, with the illustrative-cost-model caveat always visible.

7.3 Sequence: A What-If Simulation

1. User adjusts a slider in **WhatIfSliders** (e.g., RAM 3200 → 4000 MB).
2. Frontend debounces (300ms) and calls POST `/whatif` with the baseline `prediction_id` and the changed field.
3. Backend loads the baseline input from predictions, applies the delta, and re-runs inference via `services/inference.py`.
4. `services/margin_model.py` computes the estimated margin delta for the adjusted spec.
5. Backend writes a row to `whatif_simulations` and returns the new tier, probability vector, and margin delta.
6. **MarginImpactPanel** updates without a full page reload.

8. Technical Stack & Infrastructure

Component	Technologies / Frameworks Chosen	Rationale / Justification
Frontend / UI	React, Tailwind CSS, Axios, Recharts	React drives the prediction dashboard and the what-if slider playground; Recharts renders SHAP waterfall and feature-importance charts; Axios standardizes calls to FastAPI.
Backend API	FastAPI, Pydantic	Async, type-hinted framework with automatic OpenAPI docs and request validation (Pydantic), which is what enforces the API contract in Chapter 11.
ML / AI Frameworks	scikit-learn, pandas, numpy, shap, matplotlib	scikit-learn provides RFECV/RandomForestClassifier; the shap library provides TreeSHAP, which has an exact, fast implementation for tree ensembles (unlike model-agnostic SHAP or LIME).

Database & Storage	PostgreSQL (primary relational store) + PostGIS extension; versioned CSV/joblib artifacts on disk	PostgreSQL stores prediction logs, what-if simulation history, the model version registry, and auth/session tables — all relational, write-light, query-by-filter workloads it fits well. PostGIS is enabled per program technical-stack requirements and reserved for the roadmap regional-pricing-benchmark feature (Chapter 5); it is not exercised by core v1 endpoints. See ADR-003.
Deployment Platform	Render (FastAPI + managed Postgres) + Vercel (React)	Both auto-deploy from GitHub on push; Render's managed Postgres add-on avoids self-hosting a DB; splitting frontend/backend avoids one service's cold starts or static-asset limits capping the other. See ADR-005.

9. Data Architecture, Schema & Governance

9.1 PostgreSQL Schema

Table	Key Columns	Purpose
predictions	id (PK), created_at, input_json, predicted_tier, probabilities_json, model_version_id (FK)	Audit log of every /predict call; backs error analysis (Ch. 15) and observability dashboards (Ch. 18).
whatif_simulations	id (PK), prediction_id (FK), adjusted_spec_json, resulting_tier, margin_estimate, created_at	Stores each what-if slider run so PMs can revisit past simulations (Chapter 5).
model_registry	id (PK), version_tag, trained_at, cv_accuracy, cv_macro_f1, artifact_path, is_active	Tracks which trained model artifact is currently served, satisfying the Model Card's versioning requirement (Chapter 13).
users	id (PK), username, password_hash, role, created_at	Backs the scoped JWT auth layer (Chapter 10.4); single seeded analyst account for v1.
regions (PostGIS, roadmap)	id (PK), region_name, geom (geometry), avg_price_tier	Reserved for the regional price-benchmarking roadmap feature; not populated or queried in v1 (see ADR-003).

9.2 Data Governance

- **Retention:** predictions and whatif_simulations rows are retained for the duration of the OJT evaluation cycle; no automatic purge is implemented in v1.
- **Versioning:** Every served prediction is tagged with the model_registry.version_tag active at request time, so historical predictions remain attributable to a specific trained model even after retraining.
- **Privacy:** No personal or user-identifying data is stored — inputs are device specifications only; the single seeded analyst credential is stored hashed (bcrypt), never in plaintext or version control.

9.3 Dataset Reproducibility

The Mobile Price Classification dataset is ~2,000 rows across 20 specification features plus the price_range target; exact row/column counts are confirmed and recorded in the README during Month 1 EDA. Python dependencies are pinned in requirements.txt and a fixed random seed (42) is used throughout (split, RFECV, Random Forest), so a reviewer can reproduce the reported metrics exactly.

10. Functional & Non-Functional Requirements

10.1 Functional Requirements

1. **FR-01 Core prediction:** User supplies specifications; the system applies the RFE-selected feature subset and returns the predicted price-range tier via the trained Random Forest classifier.
2. **FR-02 Local explainability:** For any logged prediction, return TreeSHAP attribution values explaining the model's output for that specific input.
3. **FR-03 What-if simulation:** Accept an adjusted specification relative to a baseline prediction and return the resulting tier, probability shift, and margin estimate.
4. **FR-04 Global feature importance:** Return the RFE-selected features ranked by Random Forest importance score.
5. **FR-05 Model versioning:** Track which trained model artifact is currently active and serve predictions only from that version.
6. **FR-06 Scoped authentication:** Restrict /predict, /whatif, and /explain to an authenticated analyst session.

10.2 Non-Functional Requirements

- **Performance & Latency:** p95 inference latency < 200ms for /predict and /whatif; TreeSHAP /explain calls target p95 < 500ms given on-demand (not per-prediction) invocation.
- **Scalability & Concurrency:** The FastAPI backend handles at least 20 concurrent prediction/what-if requests without degraded latency, with the model and SHAP explainer loaded once at startup.
- **Reliability & Availability:** Graceful error handling for malformed payloads; structured logging of all requests and failures (Chapter 18).

10.3 Security & Privacy (summary — full detail in Chapter 17)

- No personally identifiable data is involved; all inputs are validated against Pydantic schemas and rejected with 422 on malformed payloads.

10.4 Auth / Session Handling (Scoped)

- Single shared login for the pricing-analyst persona (one seeded username/password), issuing a short-lived JWT stored client-side.
- FastAPI dependency checks the JWT on `/predict`, `/whatif`, and `/explain`; `/health` and `/feature-importance` remain public for CI/deployment checks and general transparency.
- No self-service signup, password reset, or multi-user roles in this scope — explicitly out of bounds unless core deliverables are complete ahead of schedule.

11. API Contract & Interface Design

Frozen before backend implementation begins, so the React frontend can build against mocked responses in parallel (Chapter 22).

Endpoint	Method	Purpose
<code>/predict</code>	POST	Accepts phone specifications, returns predicted price-range tier with class probabilities.
<code>/whatif</code>	POST	Accepts a baseline spec plus one or more adjusted fields (slider deltas); returns the new predicted tier, probability shift, and an estimated margin delta. Powers the Assisted Pricing Optimization Engine (Chapter 5).
<code>/explain/{prediction_id}</code>	GET	Returns TreeSHAP local attribution values for a previously logged prediction, used to render the waterfall chart.
<code>/feature-importance</code>	GET	Returns the RFE-selected features ranked by global Random Forest importance score.
<code>/health</code>	GET	Liveness check confirming the active model artifact (<code>model_registry</code>) is loaded; used by deployment/CI and uptime monitoring.

POST /predict — request body:

```
{ "battery_power": 1500, "ram": 3200, "px_height": 800, "px_width": 1200,
  "mobile_wt": 150, "n_cores": 4, "int_memory": 32, ... }
```

POST /predict — response body:

```
{ "prediction_id": "pred_8f21", "predicted_price_range": 2, "label":
  "Mid-High", "class_probabilities": [0.03, 0.12, 0.71, 0.14] }
```

POST /whatif — request body:

```
{ "baseline_prediction_id": "pred_8f21", "adjustments": { "ram": 4000 } }
```

POST /whatif — response body:

```
{ "predicted_price_range": 3, "label": "High", "class_probabilities": [0.01,
  0.06, 0.34, 0.59], "margin_estimate_delta": 4.20 }
```

GET /explain/{prediction_id} — response body:

```
{ "base_value": 0.25, "attributions": [ { "feature": "ram", "shap_value": 0.31
  }, { "feature": "battery_power", "shap_value": 0.14 } ] }
```

GET /feature-importance — response body:

```
{ "selected_features": [ { "name": "ram", "importance": 0.34, "rank": 1 }, {
  "name": "battery_power", "importance": 0.21, "rank": 2 } ],
  "n_features_selected": 8, "n_features_total": 20 }
```

Exact field names are finalized against the actual Mobile Price Classification dataset columns during Month 1; the shapes above are the committed contract, not the final field list.

12. Model Development Methodology

12.1 Nested Cross-Validation & Split Schema

To avoid the RFE-tuning-on-the-test-set leakage that makes single-split feature selection unreliable, model selection uses nested cross-validation: an outer loop for unbiased performance estimation and an inner loop for RFECV/hyperparameter search.

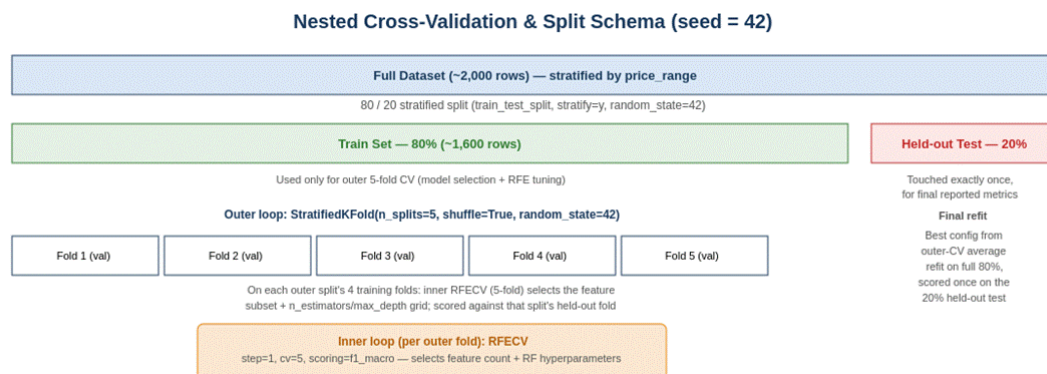


Figure 2 — Nested cross-validation and train/test split schema, fixed seed = 42.

12.2 Hyperparameter Search Space

- RFECV: step = 1, cv = 5 (inner), scoring = f1_macro, min_features_to_select tuned between 5–15.
- RandomForestClassifier: n_estimators $\in \{100, 200, 400\}$, max_depth $\in \{\text{None}, 10, 20\}$, min_samples_leaf $\in \{1, 2, 4\}$ — grid-searched within the inner loop.
- Baseline model: the same RandomForestClassifier grid, trained on all features (no RFE), for the comparison required by the committed targets (Chapter 3).

13. Model Card

Field	Details
Model Details	RandomForestClassifier trained on an RFECV-selected feature subset of the Mobile Price Classification dataset. Framework: scikit-learn. Versioned via model_registry (Chapter 9); artifact format: joblib.
Intended Use	Decision-support for product managers/pricing analysts estimating which of 4 price tiers a candidate device specification is likely to fall into, and which specifications most influence that placement. Not intended for final, unreviewed pricing decisions without human sign-off.
Out-of-Scope Uses	Not validated for price prediction outside the specification ranges present in the training dataset (e.g., foldable/flagship devices with specs far beyond the dataset's distribution); not a substitute for real market/competitor pricing data.
Factors	Evaluated across the 4 price_range classes (low/medium/high/very-high). No demographic or protected-attribute factors are present in this dataset — all features are hardware specifications.
Metrics	Accuracy, macro F1-score (primary, Chapter 3), per-class precision/recall, and a confusion matrix (Chapter 15). Macro F1 is reported alongside accuracy specifically so class-imbalance effects can't hide in an aggregate accuracy number.
Training Data	80% stratified split (~1,600 rows) of the Mobile Price Classification dataset (Kaggle), fixed seed 42. See Chapter 12 for the full CV schema.
Evaluation Data	20% held-out stratified split (~400 rows), touched exactly once for final reported metrics — never used during RFECV or hyperparameter selection.
Quantitative Analysis	Reported against the committed targets (Accuracy $\geq 90\%$, macro F1 ≥ 0.88) and the no-RFE Random Forest baseline; full breakdown in Chapter 15 (Error Analysis).
Ethical Considerations	The dataset contains only hardware specifications — no personal, demographic, or protected-attribute data — so conventional fairness-across-groups analysis does not apply. The main ethical risk is over-reliance: presenting the what-if margin estimate (Chapter 5) as precise when it is built on an illustrative, non-vendor-sourced cost model.

Caveats & Recommendations	Margin estimates from the what-if playground are approximations for demonstration, not real bill-of-materials data, and are labeled as such in the UI. Predictions for specification combinations far outside the training distribution should be treated as low-confidence; the UI surfaces the model's class probabilities so analysts can judge confidence themselves rather than trusting a bare label.
---------------------------	---

14. Explainability Design (TreeSHAP)

TreeSHAP is integrated at the `services/shap_service.py` layer (Chapter 7.1) using the `shap` library's `TreeExplainer`, which computes exact Shapley values for tree ensembles in polynomial time — a meaningful advantage over sampling-based KernelSHAP or LIME for this use case (see ADR-004, Chapter 16).

14.1 Consistency Checks

- **Additivity check:** for every `/explain` response, the sum of SHAP attribution values plus the base value must reconstruct the model's raw output for that class within a small numerical tolerance — enforced as an automated test (Chapter 19).
- **Global-local agreement:** the sign and rough ordering of top SHAP features for a batch of predictions is spot-checked against the global feature-importance ranking (Chapter 10.1) during Viva 2 to catch any systematic mismatch.

14.2 Presentation

The `React ShapWaterfall` component (Chapter 7.2) renders attributions as a waterfall from the base value to the final predicted probability, ordered by absolute contribution, so a PM can see at a glance which one or two specifications dominate the decision.

15. Error Analysis & Failure Mode Assessment

Error analysis is a required deliverable, not an afterthought — it directly informs the Model Card's caveats (Chapter 13) and is logged against real predictions stored in the predictions table (Chapter 9).

15.1 Confusion Matrix Analysis

- A full 4x4 confusion matrix is generated on the held-out test set at every Viva 2/3 checkpoint, alongside per-class precision/recall.
- **Boundary-class focus:** price-tier classes are ordinal (low → very-high), so misclassifications between adjacent tiers (e.g., Medium predicted as Mid-High) are treated as materially less severe than misclassifications across two or more tiers (e.g., Low predicted as Very-High); both rates are reported separately.

15.2 Feature-Level Error Breakdown

- For misclassified rows, the top SHAP attribution (Chapter 14) is inspected to identify whether errors cluster around specific specifications (e.g., devices with unusually high RAM but low camera resolution) — informing whether the RFE-selected feature set is missing a signal.

15.3 Known Failure Modes

- Out-of-distribution specs: combinations far outside the training data's range (flagged when nearest-neighbor distance in feature space exceeds a threshold) are expected to have lower reliability; the UI surfaces class probabilities so low-confidence cases are visible rather than hidden behind a single label.
- What-if extrapolation: the what-if playground allows slider values beyond the observed training range; predictions in that region are explicitly flagged in the UI as extrapolated.

16. Architecture Decision Records (ADRs)

Five ADRs cover the decisions with the largest downstream impact on this project, addressing the “missing ADRs” gap directly.

ADR	Decision	Alternatives Considered	Status
ADR-001	RFE (wrapped around Random Forest) as the feature-selection method	Mutual information; L1 (Lasso); no selection	Accepted
ADR-002	FastAPI as the backend framework	Flask; Django REST Framework	Accepted
ADR-003	PostgreSQL + PostGIS as the relational store	SQLite; MongoDB; no DB (files only)	Accepted
ADR-004	TreeSHAP for local explainability	LIME; permutation importance only	Accepted
ADR-005	Split deployment: Render (backend+DB) + Vercel (frontend)	Single Render/Railway service hosting both	Accepted

ADR-001: Feature Selection Method

Context:

The project needs a feature-selection approach that both improves/matches model performance and produces an interpretable ranking for the feature-importance report and Model Card.

Decision:

Use Recursive Feature Elimination wrapped around a Random Forest estimator (RFECV for the final tuned count), grounded in the JMLR 2003 feature-selection survey. Mutual information and L1 (Lasso) are retained as a cross-check against the alternatives listed above, but not as the primary method.

Consequences:

RFE is more compute-intensive than a single mutual-information pass, but ties feature selection directly to the same model family used for prediction, and nested CV (Chapter 12) controls for its main failure mode (selection instability).

ADR-002: Backend Framework

Context:

Needed a Python backend framework that serves a scikit-learn model with low latency, validates requests against the API contract (Chapter 11), and auto-generates API docs for the mentor/evaluator to inspect.

Decision:

FastAPI, with Pydantic models enforcing the request/response schema.

Consequences:

Async support and auto-generated OpenAPI docs speed up both development and evaluation; the team accepts a smaller plugin ecosystem than Django in exchange for this fit.

ADR-003: Relational Data Store

Context:

The program's master technical-stack template requires PostgreSQL/PostGIS. The core v1 workload (prediction logs, what-if history, model registry, single-role auth) is relational but not geospatial.

Decision:

Adopt PostgreSQL as the primary store for all five tables (Chapter 9) and enable the PostGIS extension, with a dedicated (currently unpopulated) regions table reserved for a roadmap regional-price-benchmarking feature. This satisfies the program requirement honestly — PostGIS is present and reserved for a concretely scoped future feature — rather than adding geospatial data with no real use in v1.

Consequences:

Slightly more operational overhead than SQLite for a project this size, offset by using Render's managed Postgres add-on rather than self-hosting; PostGIS costs nothing extra to enable and unblocks the regional-benchmarking roadmap item without a later migration.

ADR-004: Explainability Method

Context:

PMs need to understand individual predictions (not just global feature importance) to trust and act on the what-if playground's recommendations.

Decision:

Use TreeSHAP (via the shap library's TreeExplainer) rather than LIME or permutation importance alone.

Consequences:

TreeSHAP is exact and fast for tree ensembles specifically (versus LIME's local sampling approximation), at the cost of being tied to tree-based models — acceptable since Random Forest is the committed model family (ADR-001).

ADR-005: Deployment Topology

Context:

Needed a deployment plan that supports a public live link (Final Deliverables milestone) on free/low-cost tiers, without one component's limits (cold starts, static-asset serving) capping the other.

Decision:

Split deployment: Render hosts the FastAPI backend and managed PostgreSQL; Vercel hosts the static React build. Both auto-deploy from GitHub on push.

Consequences:

Two services to configure instead of one, but each is used for what it's best at, and both integrate cleanly with the GitHub Actions CI/CD pipeline (Chapter 21).

17. Security & Privacy Design

- **Input validation:** every request body is validated against Pydantic schemas (Chapter 11); malformed or out-of-range values return 422, never silently coerced.
- **Authentication:** short-lived JWTs issued on login, verified via a FastAPI dependency on `/predict`, `/whatif`, `/explain` (Chapter 10.4); tokens are never logged.
- **Credential storage:** the single seeded analyst password is stored bcrypt-hashed in the users table (Chapter 9); never committed to version control.
- **Secrets management:** database URL, JWT signing secret, and any API keys are supplied via environment variables (Render/Vercel secret stores), never hard-coded.
- **Dependency hygiene:** Python and JS dependencies are pinned (`requirements.txt` / `package-lock.json`); `pip audit` / `npm audit` run in CI (Chapter 21) to catch known vulnerabilities before deployment.
- **Data privacy:** no personal or user-identifying data is collected or stored anywhere in the system; the dataset and all logged predictions are device specifications only.

18. Observability & Monitoring

- **Structured logging:** every request to `/predict`, `/whatif`, and `/explain` logs a JSON line (timestamp, endpoint, latency_ms, status_code, model_version) to stdout, captured by Render's log stream.
- **Metrics:** request count, p50/p95/p99 latency, and error rate per endpoint are derived from the structured logs; prediction class distribution is tracked over time as a lightweight drift signal against the training distribution.
- **Error tracking:** unhandled exceptions are logged with a stack trace and a correlation id that matches the client-visible error response, so a specific failed request can be traced end-to-end.
- **Alerting (manual for v1):** `/health` is polled by an uptime check (e.g., UptimeRobot free tier); a failing check is the trigger to inspect logs — automated alerting integrations are a stretch goal beyond core scope.

19. Testing, QA & Validation Strategy

Test Layer	Tooling / Approach	Coverage Target
Unit tests	pytest — preprocessing functions, feature-selection wrapper, cost/margin model in isolation	≥ 80% line coverage on app/services/
API contract tests	pytest + httpx against a running FastAPI test client; validates request/response shapes in Chapter 11 for all 5 endpoints	Every endpoint has at least one happy-path and one validation-error test
Model validation	Nested CV (Chapter 12); assert CV accuracy/macro F1 do not regress below committed targets before a new model_registry entry is marked active	Automated gate in CI before deployment
Frontend tests	React Testing Library for the prediction form, slider playground, and chart components	Core user flows (submit spec → see prediction → adjust slider → see updated result)
Edge cases	Out-of-range / missing / malformed specification payloads; SHAP explainer called on an unseen model version	Documented failure modes return structured 4xx errors, never a 500

19.1 Edge Case Handling

- Out-of-range or missing specification inputs (e.g., negative battery power) return a structured 422 rather than a silent failure or a 500.

- `/explain` called against a `prediction_id` from a superseded `model_registry` version returns a clear 409 rather than a misleading explanation from the wrong model.

20. Requirements Traceability Matrix

Every business need identified in Chapters 2–5 traces to a shipped functional requirement, a concrete design component or database entity, and at least one test case — closing the traceability gap flagged in review.

Req ID	Business Need	Functional Requirement	Design Component / DB Entity	Test Case ID
BR-01	Predict a device's price tier from specs	FR-01 Core prediction (Ch. 10.1)	POST <code>/predict</code> ; predictions table	TC-API-01, TC-API-02
BR-02	Explain which specs drove a specific prediction	FR-02 Local explainability (Ch. 14)	GET <code>/explain/{id}</code> ; TreeSHAP service	TC-API-03, TC-SHAP-01
BR-03	Simulate spec trade-offs and their price/margin impact	FR-03 What-if playground (Ch. 5)	POST <code>/whatif</code> ; <code>whatif_simulations</code> table; React slider UI	TC-API-04, TC-UI-01
BR-04	See which specs matter most overall, not just per-prediction	FR-04 Global feature importance (Ch. 10.1)	GET <code>/feature-importance</code>	TC-API-05
BR-05	Trust that predictions come from a verified, current model	FR-05 Model versioning (Ch. 9, 13)	<code>model_registry</code> table; GET <code>/health</code>	TC-API-06, TC-CI-01
BR-06	Restrict access to authorized analysts	FR-06 Scoped auth (Ch. 10.4)	<code>users</code> table; JWT dependency	TC-AUTH-01, TC-AUTH-02
BR-07	Detect and understand model mistakes	FR-07 Error analysis (Ch. 15)	<code>predictions</code> table (logged outcomes); <code>confusion-matrix</code> report	TC-EVAL-01, TC-EVAL-02

21. Deployment & CI/CD Strategy

- **Version Control Strategy:** Git branching model with ``main``, ``development``, and feature branches (e.g., ``feature/rfe-pipeline``, ``feature/whatif-api``, ``feature/shap-explainer``); each feature branch merges only after its own automated tests pass, standing in for peer review on this solo project.
- **Containerization:** Dockerize the FastAPI backend for reproducible local and deployment environments; React build deploys as static assets to Vercel.

- **CI/CD & Live Link:** GitHub Actions runs unit, contract, and model-validation tests (Chapter 19) on every push/PR, including the dependency-audit step (Chapter 17); on merge to `main`, Render auto-deploys the backend + Postgres and Vercel auto-deploys the frontend, keeping the public live link current ahead of the Final Deliverables milestone.

22. Risk Management & Constraints

Identified Risk	Impact / Severity	Mitigation Strategy
Compute / GPU resource limits	Low–Medium	Random Forest + RFE training is CPU-friendly; use free-tier compute (Colab/Kaggle) if nested-CV grid search is slow, or sub-sample during development.
Class imbalance across the 4 price-range tiers	Medium	Verify class balance during EDA; apply stratified sampling (already used), class-weighting, or SMOTE if imbalance exists.
Feature-selection instability (RFE ranking sensitive to RF randomness)	Medium	Fixed seeds, RFECV nested CV (Ch. 12), cross-check against mutual information / L1 as a stability sanity-check.
API/frontend integration delays	Medium	API contract (Ch. 11) frozen before backend work starts; frontend built against mocked responses; contract tests added before Viva 3.
Target metrics not met (Accuracy/macro F1 below committed thresholds)	Medium	If Viva 2 checkpoint falls short, widen the RFE feature count or retune RF hyperparameters before declaring the target unreachable; document the gap in the Model Card if still unmet.
TreeSHAP compute cost at request time	Low–Medium	TreeSHAP is fast for tree ensembles, but /explain is served on-demand (not on every /predict call) and results are cached in the predictions table to avoid recomputation.
Margin/cost model is illustrative, not real BOM data	Medium	Explicitly labeled as an approximation in the UI and Model Card (Ch. 5, 13); documented as a caveat rather than presented as authoritative.
Solo-contributor bandwidth against a 3-month timeline with an expanded scope	Medium–High	Explicit sequencing in Chapter 1 (Solo Execution Plan); what-if/SHAP work is scoped as an extension layered on top of a working core classifier+API, so a functioning fallback exists even if the enhancement slips.

23. Project Milestones & Evaluation Roadmap

Phase / Evaluation	Milestones & Deliverables	Deadline
PRD Approval Phase	Finalized PRD (this document) submitted, reviewed, and approved by Mentor.	30th Aug – 3rd Sept
GitHub Base Setup	Repository setup, architecture layout, baseline project structure, ADR log seeded.	6th September
Viva 1 (Month 1)	– Study the grounding paper and RFE + Random Forest fundamentals. – Scaffold FastAPI backend, Postgres schema (Ch. 9), and a basic React shell. – Acquire/explore the dataset; confirm row/column counts for the Model Card. – Build a baseline (no-RFE) model pipeline as the comparison floor.	15th – 22nd Sept
Viva 2 (Month 2)	– Train/tune the RFECV + Random Forest model per the CV schema (Ch. 12) against committed targets. – Expose /predict, /feature-importance, /health per the frozen API contract. – Integrate TreeSHAP and ship /explain/{id}. – Build core React screens (prediction form, feature-importance chart).	15th – 22nd Oct
Viva 3 (Month 3)	– Ship the what-if playground (/whatif + slider UI + margin estimate). – Complete error analysis (Ch. 15) and finalize the Model Card (Ch. 13). – Finish all 5 ADRs and the traceability matrix (Ch. 16, 20). – Deploy, write the README/architecture docs, and prepare the demo.	15th – 22nd Nov
Final Deliverables	Public live deployment link and project demonstration video submission.	25th November
External Final Viva	Comprehensive project defense, code review, and final demonstration.	15th – 22nd Dec

24. Definition of Done

Each deliverable is only considered complete when it meets the bar below — this is what mentor and external-viva sign-off is checked against.

Deliverable	Definition of Done
Feature-selected classifier	Trained per the nested-CV schema (Ch. 12), meets or exceeds committed Accuracy/macro F1 targets on the untouched 20% test set, and the selected feature subset is fixed and version-controlled in <code>model_registry</code> .
Prediction + what-if API	All 5 endpoints (Ch. 11) match their schemas, return correct status codes on invalid input, and are covered by passing contract tests.
Explainability (TreeSHAP)	<code>/explain/{id}</code> returns attribution values that sum consistently with the model's output margin (SHAP additivity check passes in tests).
What-if playground	A PM can move any spec slider and see an updated tier + probability shift + margin estimate in under 1 second, with the illustrative-cost-model caveat visible in the UI.
Model Card & ADRs	Model Card (Ch. 13) reflects the actually-deployed <code>model_registry</code> entry; all 5 ADRs (Ch. 16) are status = Accepted or explicitly superseded.
Traceability matrix	Every business need (BR-01...BR-07) maps to at least one shipped functional requirement, design component, and passing test case with no blank cells.
Full-stack app (React + FastAPI + Postgres)	Deployed to a public live link; a first-time visitor can log in, submit a spec, see a prediction with explanation, and run a what-if simulation end-to-end with no console errors.
README & architecture diagram	A reviewer with no prior context can clone the repo, follow setup steps, and reproduce the reported metrics without contacting the author.

25. Grounding References & Appendices

25.1 References

- Paper: An Introduction to Variable and Feature Selection — Journal of Machine Learning Research (2003).
- Dataset: Mobile Price Classification (Kaggle).

25.2 Glossary

- RFE / RFECV: Recursive Feature Elimination (with cross-validated feature-count selection) — iteratively removes the least-important feature(s) and refits, per the grounding paper's methodology.
- TreeSHAP: An exact, efficient algorithm for computing Shapley-value feature attributions for tree-based models.
- ADR: Architecture Decision Record — a short document capturing the context, decision, and consequences of a significant technical choice (Chapter 16).
- Model Card: A standardized, structured summary of a trained model's intended use, performance, training/evaluation data, and limitations (Chapter 13).
- Macro F1-score: The unweighted average of per-class F1 scores, chosen so no single price-tier class's performance is hidden by class imbalance.